컴퓨터 비전

3주차

목차

- **이벤트 처리**

- **그리기**
  - 기본 도형 그리기
  - 글자 쓰기
  - 원 그리기
  - 타원 그리기

- **이벤트 & 그리기**

- **영상 파일 처리**
  - 영상 파일 읽기
  - 영상 파일 저장

- **과제**

키보드 이벤트 처리

- **이벤트**
  - 프로그램에 의해 감지되고 처리될 수 있는 동작이나 사건
  - 예
    - 사용자가 키보드의 키를 누르는 것
    - 마우스를 움직인다거나 마우스 버튼을 누르는 것
    - 깊이 들어가면 타이머(timer)와 같은 하드웨어 장치가 발생시키는 이벤트
    - 사용자가 자체적으로 정의하는 이벤트

- **일반적으로 이벤트를 처리하기 위해 콜백(callback) 함수 사용**

- **콜백 함수**
  - 개발자가 시스템 함수를 직접 호출하는 방식
  - 이벤트가 발생하거나 특정 시점에 도달했을 때 시스템이 개발자가 등록한 함수 호출

- **OpenCV에서도 기본적인 이벤트 처리 함수 지원**
  - 키보드 이벤트, 마우스 이벤트, 트랙바(trackbar) 이벤트

키보드 이벤트 처리

- 키보드 이벤트 처리 코드

```
switch_case = {
    ord('a'): "a키 입력",
    ord('b'): "b키 입력",
    0x41: "A키 입력",
    int('0x42', 16): "B키 입력",
    2424832: "왼쪽 화살표키 입력",
    2490368: "윗쪽 화살표키 입력",
    2555904: "오른쪽 화살표키 입력",
    2621440: "아래쪽 화살표키 입력"
}

image = np.zeros((480, 640), np.float)
cv2.namedWindow('Keyboard Event')
cv2.imshow("Keyboard Event", image)
```

화살표키 등 특수키 인지 위해 사용

```
while True:
    key = cv2.waitKeyEx(100)
    if key == 27:
        break

    try:
        result = switch_case[key]
        print(result)

    except KeyError:
        result = -1

cv2.destroyAllWindows()
```

마우스 이벤트 처리

- **마우스 이벤트 처리 콜백 함수**

onMouse(event, x, y, flags, param=None)

■ 설명: 발생한 마우스 이벤트에 대한 처리와 제어를 구현하는 콜백 함수. cv2.setMouseCallback() 함수의 두 번째 인수(onMouse)의 구현부, 따라서 이름이 같아야 함. onMouse() 함수의 인수 구조(인수 타입, 인수 순서 등)를 유지해야 함.

인수 설명

■ event 발생한 마우스 이벤트의 종류

■ x, y 이벤트 발생 시 마우스 포인터의 x, y 좌표

■ flags 마우스 버튼과 동시에 특수키([Shift], [Alt], [Ctrl])를 눌렀는지 여부 확인

옵션	값	설명
cv2.EVENT_FLAG_LBUTTON	1	왼쪽 버튼 누르기
cv2.EVENT_FLAG_RBUTTON	2	오른쪽 버튼 누르기
cv2.EVENT_FLAG_MBUTTON	4	중간 버튼 누르기
cv2.EVENT_FLAG_CTRLKEY	8	[Ctrl] 키 누르기
cv2.EVENT_FLAG_SHIFTKEY	16	[Shift] 키 누르기
cv2.EVENT_FLAG_ALTKEY	32	[Alt] 키 누르기

■ param 콜백 함수로 전달하는 추가적인 사용자 정의 인수

마우스 이벤트 처리

- 마우스 이벤트 종류

〈표 4.2.1〉 마우스 이벤트 종류

옵션	값	설명
cv2.EVENT_MOUSEMOVE	0	마우스 움직임
cv2.EVENT_LBUTTONDOWN	1	왼쪽 버튼 누르기
cv2.EVENT_RBUTTONDOWN	2	오른쪽 버튼 누르기
cv2.EVENT_MBUTTONDOWN	3	중간 버튼 누르기
cv2.EVENT_LBUTTONUP	4	왼쪽 버튼 떼기
cv2.EVENT_RBUTTONUP	5	오른쪽 버튼 떼기
cv2.EVENT_MBUTTONUP	6	중간 버튼 떼기
cv2.EVENT_LBUTTONDBLCLK	7	왼쪽 버튼 더블클릭
cv2.EVENT_RBUTTONDBLCLK	8	오른쪽 버튼 더블클릭
cv2.EVENT_MBUTTONDBLCLK	9	중간 버튼 더블클릭
cv2.EVENT_MOUSEWHEEL	10	마우스 휠
cv2.EVENT_MOUSEHWHEEL	11	마우스 가로 휠

함수 설명		
def setMouseCallback(windowName, onMouse, param=None) → None ■ 설명: 사용자가 정의한 마우스 콜백 함수를 시스템에 등록		
인수 설명	■ winname	이벤트 발생을 확인할 윈도우 이름, 문자열
	■ onMouse	마우스 이벤트를 처리하는 콜백 함수 이름(콜백함수)
	■ param	이벤트 처리 함수로 전달할 추가적인 사용자 정의 인수

마우스 이벤트 처리

- 예제 코드

```
def onMouse(event, x, y, flags, param):
    if event == cv2.EVENT_LBUTTONDOWN:
        print("마우스 왼쪽 버튼 누르기")
    elif event == cv2.EVENT_RBUTTONDOWN:
        print("마우스 왼쪽 버튼 누르기")
    elif event == cv2.EVENT_LBUTTONUP:
        print("마우스 왼쪽 버튼 떼기")
    elif event == cv2.EVENT_RBUTTONUP:
        print("마우스 왼쪽 버튼 떼기")

image = np.zeros((480, 640), np.uint8)
```

```
title1 = "Mouse Event1"
title2 = "Mouse Event2"

cv2.imshow(title1, image)
cv2.imshow(title2, image)

cv2.setMouseCallback(title1, onMouse)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

트렉바 이벤트 처리

- **트랙바(trackbar)**
  - 일정한 범위에서 특정한 값을 선택할 때 사용하는 일종의 스크롤바 혹은 슬라이더바

cv2.createTrackbar(trackbarname, winname, value count, onChange) → None

■ 설명: 트랙바를 생성한 후, 지정한 윈도우에 추가하는 함수이다.

인수 설명	■ trackbarname	윈도우에 생성되는 트랙바 이름
	■ winname	트랙바의 부모 윈도우 이름(트랙바 이벤트 발생을 체크하는 윈도우)
	■ value	트랙바 슬라이더의 위치를 반영하는 값(정수)
	■ count	트랙바 슬라이더의 최댓값, 최솟값은 항상 0
	■ onChange	트랙바 슬라이더의 값이 변경될 때 호출되는 콜백 함수

onChange(pos) → None

■ 설명: 트랙바 슬라이더의 위치가 변경될 때마다 호출되는 콜백 함수. cv2.createTrackbar()의 마지막 인수와 이름이 같아야 한다.

인수 설명	■ pos	트랙바 슬라이더 위치

cv2.getTrackbarPos(trackbarname, winname) → retval

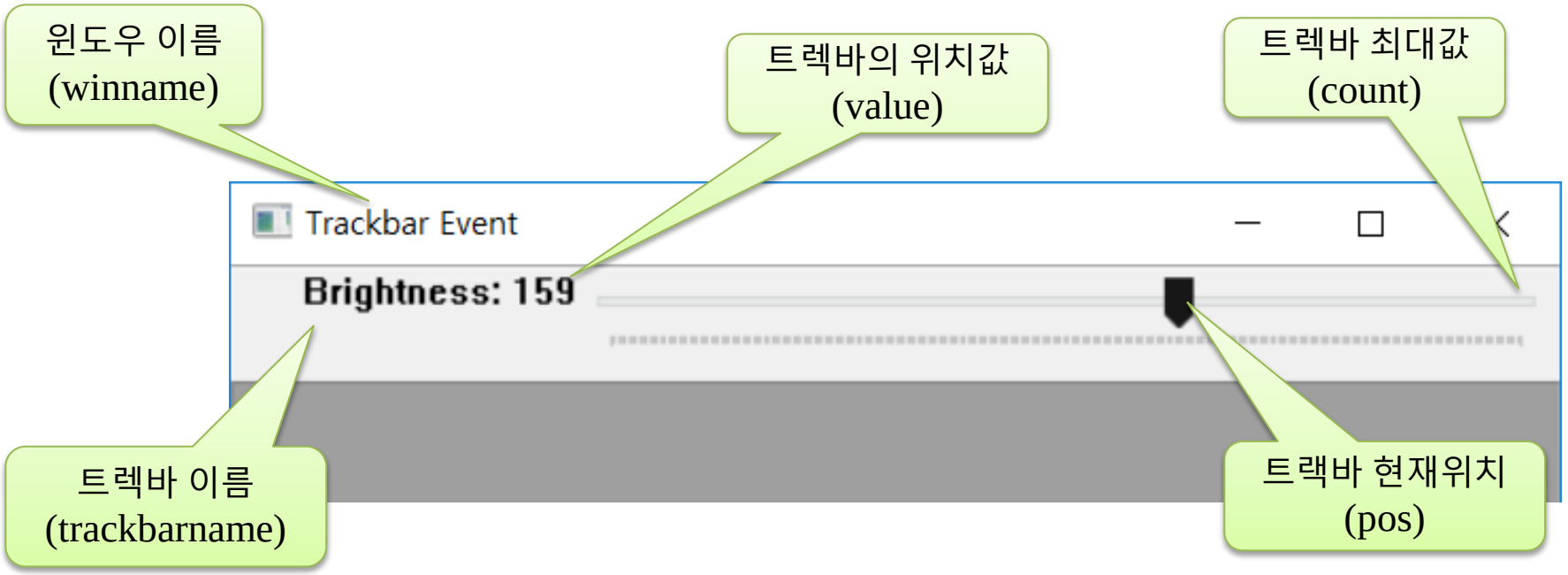
■ 설명: 지정한 트랙바의 슬라이더 위치를 반환한다.

cv2.setTrackbarPos(trackbarname, winname, pos) → None

■ 설명: 지정한 트랙바의 슬라이더 위치를 설정한다.

트렉바 이벤트 처리

- **createTrackbar(trackbarname , winname, value , count , onChange , userdata);**



트렉바 이벤트 처리

- **트렉바 이벤트 처리 예제**
  - 트렉바의 값과 전체 화소의 값과 연동

```
def onChange(value):
    global image, title

    add_value = value - int(image[0][0])
    print("추가 화소 값:", add_value)

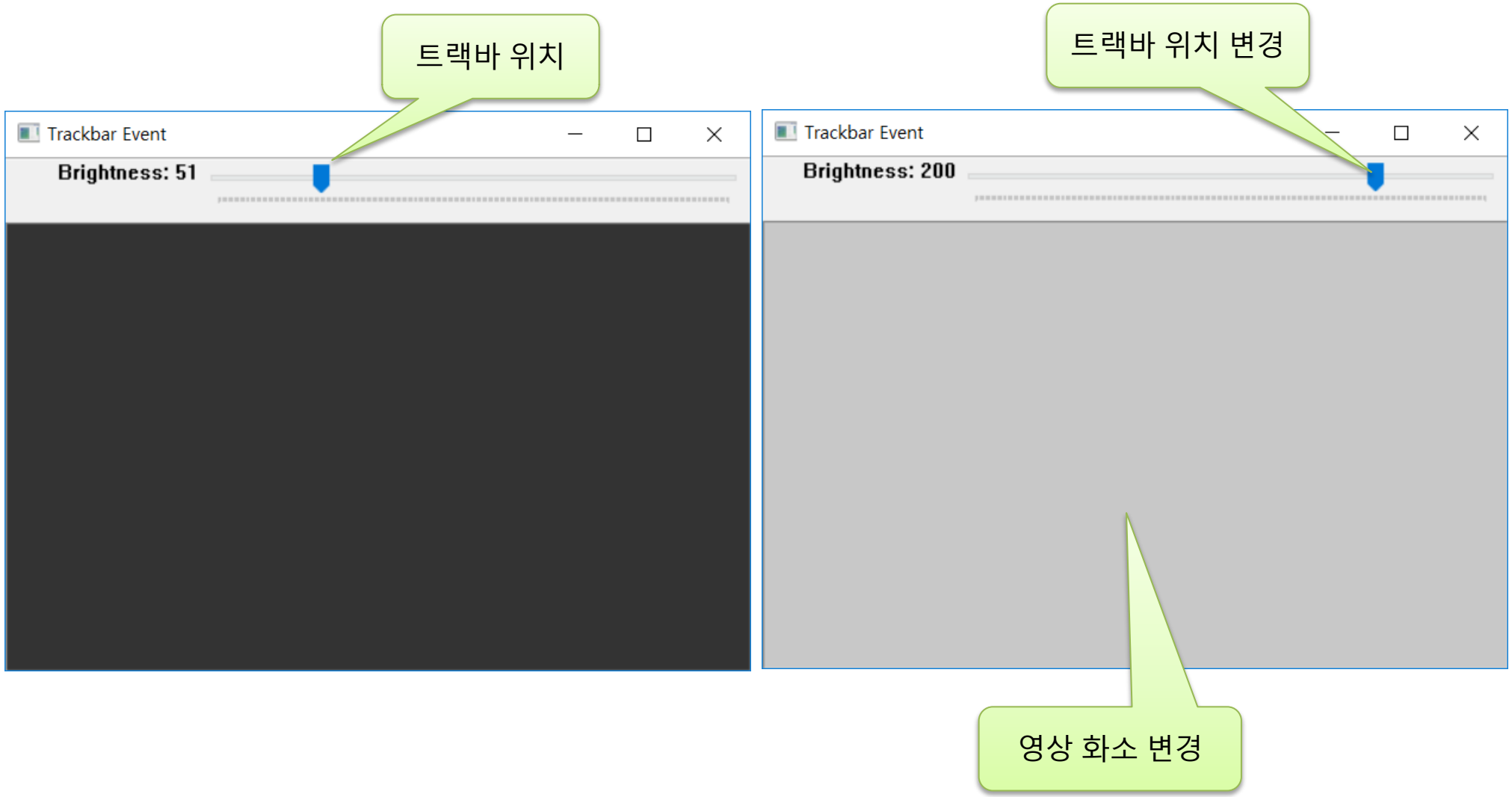
    image = image + add_value
    cv2.imshow(title, image)

image = np.zeros((480, 640), np.uint8)
title = "Trackbar Event"
cv2.imshow(title, image)

cv2.createTrackbar('Brightness', title, image[0][0], 255, onChange)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

트렉바 이벤트 처리

- 실행결과



실습

- **트렉바 + 마우스 이벤트 처리 실습**
  - 트렉바 이벤트 처리 예제에서 마우스 이벤트 처리 코드 추가
  - 마우스 우클릭, 좌클릭 이벤트 추가
    - 우클릭 : 화소 값 + 10
    - 좌클릭 : 화소 값 - 10

```
def onChange(value):...

def onMouse(event, x, y, flags, param):
    global image, bar_name

    if event == cv2.                    :
        if( image[0][0] < 246):
                                        
        cv2.setTrackbarPos(bar_name, title, image[0][0])
        cv2.imshow(title, image)

    if event == cv2.                    :
        if (image[0][0] >= 10):
            image = image - 10
        cv2.setTrackbarPos(bar_name, title, image[0][0])
        cv2.                            )
```

```
image = np.zeros((480, 640), np.uint8)

title = "Trackbar & Mouse Event"
bar_name = 'brightness'



cv2.imshow(title, image)

cv2.                    (bar_name, title, image[0][0], 255, onChange)
cv2.setMouseCallback(title, onMouse)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

과제

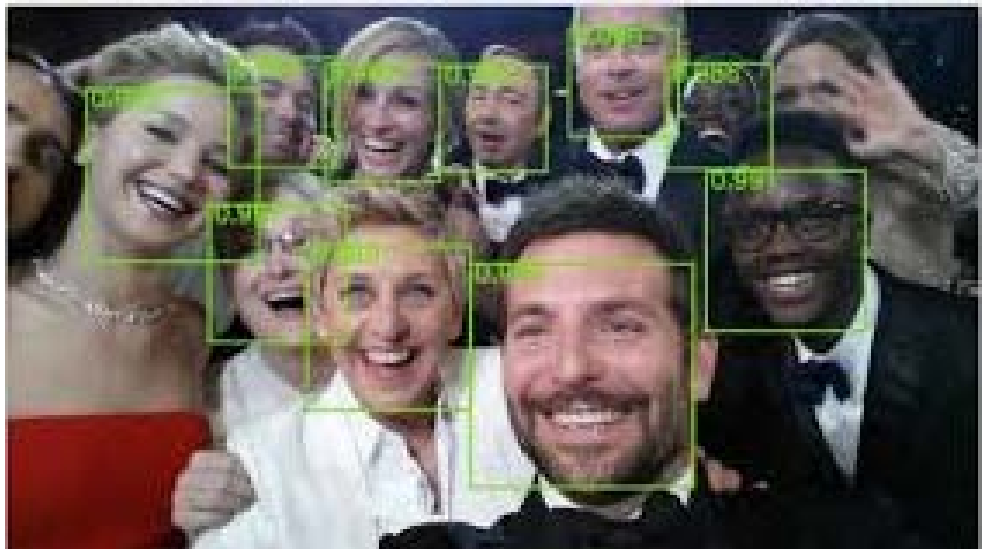
- **트렉바 + 마우스 +키보드 이벤트 처리 실습**
  - 트렉바 + 마우스 이벤트 처리 실습에서 키보드 이벤트 처리 코드 추가
  - 키보드 왼쪽 화살표, 오른쪽 화살표, 종료 q(Q) 이벤트 추가
    - 오른쪽 화살표 : 화소 값 + 10
    - 왼쪽 화살표 : 화소 값 – 10
    - q/Q : 종료

  - Hint1
    - 34p + 45p

  - Hint2
    - 키보드는 Callback 함수가 모두 호출 된 이후에 필요
    - 두 Callback 함수 호출 이전에 키보드 관련 함수 지정 시 오동작
    - 직전 실습 코드에서 키보드 관련 함수 라인에서 수정 필요

기본 도형 그리기

- **영상처리 프로그래밍 과정에서 해당 알고리즘 적용시 결과 확인 필요**
  - 얼굴 검출 알고리즘을 적용했을 때,
    - 전체 영상 위에 검출한 얼굴 영역을 사각형이나 원으로 표시
  - 차선 확인하고자 직선 검출 알고리즘을 적용했을 때,
    - 차선을 정확하게 검출했는지 확인하기 위해 도로 영상 위에 선으로 표시





기본 도형 그리기

- 직선 및 사각형 그리기 코드

```
image = np.zeros((400,600,3), np.uint8)
image[:] = (255, 255, 255)

pt1 = (50, 50)
pt2 = (250, 150)
pt3 = (400, 150)
pt4 = (500, 50)
roi = (50, 200, 200, 100)
```

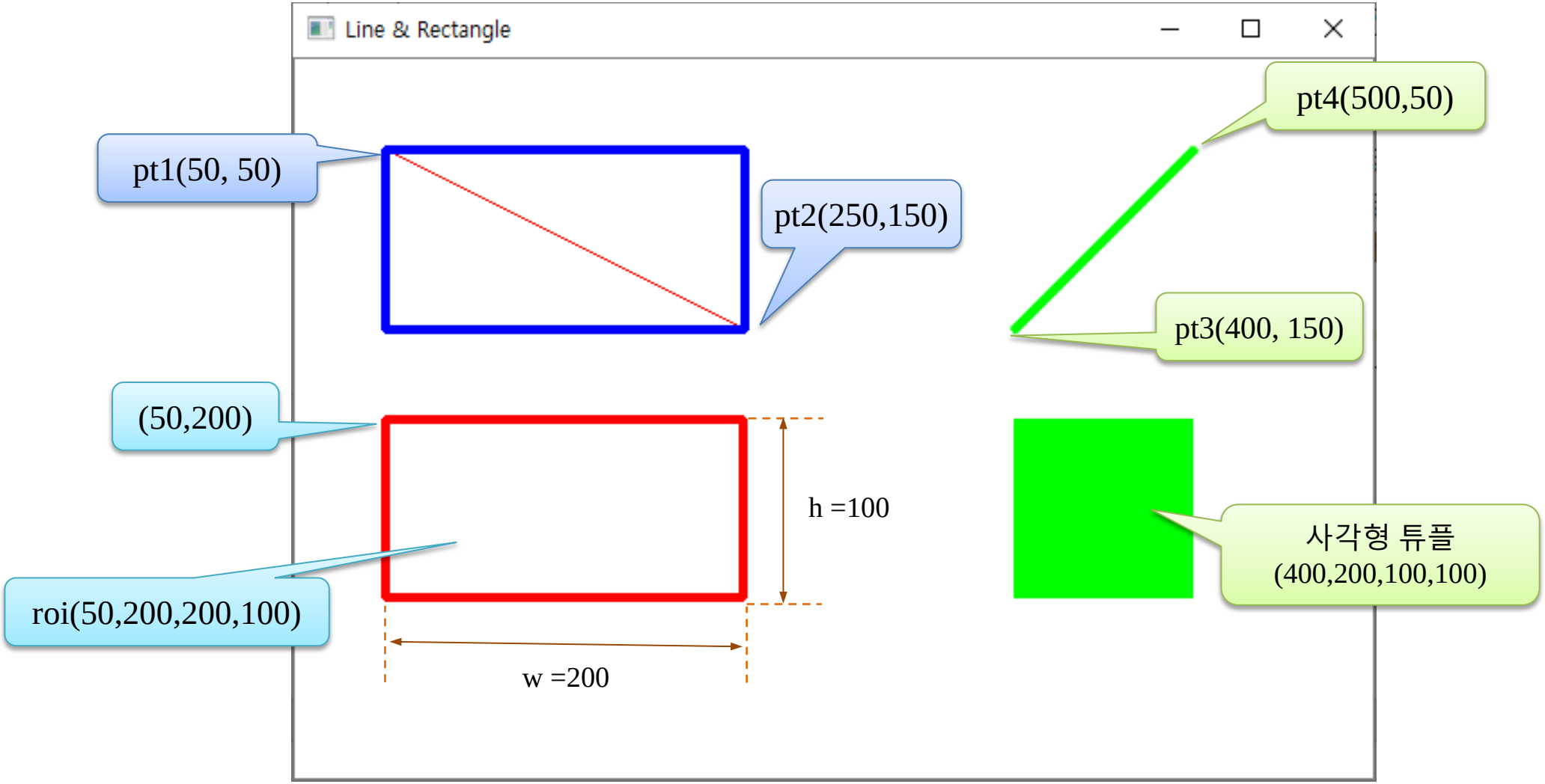
```
## line
cv2.line(image, pt1, pt2, (0, 0, 255))
cv2.line(image, pt3, pt4, (0, 255, 0))

## rectangle
cv2.rectangle(image, pt1, pt2, (255,0,0), 3)
cv2.rectangle(image, roi, (0,0,255), 3)
cv2.rectangle(image, (400, 200, 100, 100), (0,255,0), cv2.FILLED)

cv2.imshow("Line & Rectangle", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

기본 도형 그리기

- 실행결과



글자 그리기

- **cv2.putText()**
  - 행렬의 특정 위치에 원하는 글자를 써서 영상으로 표시하고 싶을 때 사용
- **형식**



글자 그리기

- 글자 그리기 함수

함수 설명		
cv2.putText(img, text, org, fontFace, fontScale, color[, thickness[, lineType[, bottomLeftOrigin]]]) → img ■ 설명: text 문자열을 org 좌표에 color 색상으로 그린다.		
인수 설명	■ img	문자열을 작성할 대상 행렬(영상)
	■ text	작성할 문자열
	■ org	문자열의 시작 좌표, 문자열에서 가장 왼쪽 하단을 의미
	■ fontFace	문자열의 폰트
	■ fontScale	글자 크기 확대 비율
	■ color	글자의 색상
	■ thichness	글자의 굵기
	■ lineTpye	글자 선의 형태
	■ bottomLeftOrigin	영상의 원점 좌표 설정(True- 좌하단 왼쪽, False- 좌상단)

글자 그리기

- **글자 폰트**
  - 세리프 체
    - 글자의 획 끝에 날카롭게 튀어나온 글자체
    - 명조체, 궁서체 등
  - 산세리프 체
    - 날카로운 장식선이 없는 글자체
    - 돋움체 , 고딕체

〈표 4.3.1〉 문자열의 폰트(fontFace)에 대한 옵션과 의미

옵션	값	설명
cv2.FONT_HERSHEY_SIMPLEX	0	중간 크기 산세리프 폰트
cv2.FONT_HERSHEY_PLAIN	1	작은 크기 산세리프 폰트
cv2.FONT_HERSHEY_DUPLEX	2	2줄 산세리프 폰트
cv2.FONT_HERSHEY_COMPLEX	3	중간 크기 세리프 폰트
cv2.FONT_HERSHEY_TRIPLEX	4	3줄 세리프 폰트
cv2.FONT_HERSHEY_COMPLEX_SMALL	5	COMPLEX 보다 작은 크기
cv2.FONT_HERSHEY_SCRIPT_SIMPLEX	6	필기체 스타일 폰트
cv2.FONT_HERSHEY_SCRIPT_COMPLEX	7	복잡한 필기체 스타일
cv2.FONT_ITALIC	16	이탤릭체를 위한 플래그

글자 그리기

- 글자 그리기 코드

```
image = np.zeros((350,500,3), np.uint8)
image[:] = (255, 255, 255)

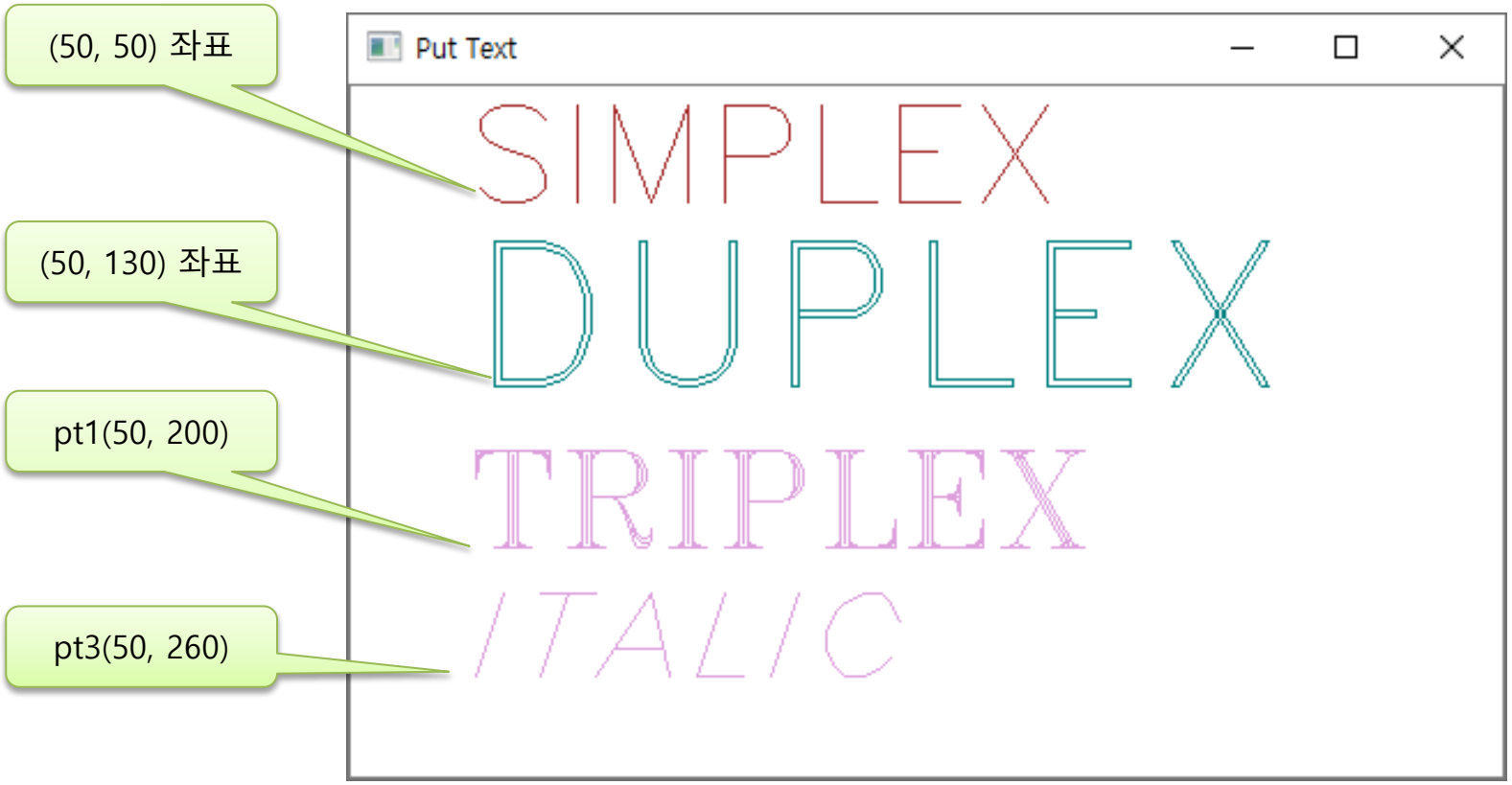
pt1 = (50, 200)
pt2 = (50, 260)

cv2.putText(image, 'SIMPLEX', (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 2, (42, 42, 165))# brown
cv2.putText(image, 'DUPLEX', (50, 130), cv2.FONT_HERSHEY_DUPLEX, 3, (128, 128, 0))# olive
cv2.putText(image, 'TRIPLEX', pt1, cv2.FONT_HERSHEY_TRIPLEX, 2, (221, 160, 221))# violet
fontFace = cv2.FONT_HERSHEY_PLAIN | cv2.FONT_ITALIC
cv2.putText(image, 'ITALIC', pt2, fontFace, 4, (221, 160, 221))# violet

cv2.imshow("Put Text", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

글자 그리기

- 실행결과


(50, 50) 좌표
(50, 130) 좌표
pt1(50, 200)
pt3(50, 260)
Put Text
SIMPLEX
DUPLEX
TRIPLEX
ITALIC

원 그리기

- **cv2.circle() 함수**
  - 원의 중심 좌표(center), 반지름(radius), 선의 색상(color)은 반드시 지정

함수 설명		
cv2.circle(img, center, radius, color[, thickness[, lineType[, shift]]]) → img ■ 설명: center를 중심으로 radius 반지름의 원을 그린다.		
인수 설명	■ img	원을 그릴 대상 행렬(영상)
	■ center	원의 중심 좌표
	■ radius	원의 반지름
	■ color	선의 색상
	■ thickness	선의 두께
	■ lineType	선의 형태 , cv2.line() 함수의 인수와 동일
	■ shift	좌표에 대한 비트 시프트 연산

원 그리기

- 원 그리기 코드

```
image = np.zeros((350,500,3), np.uint8)
image[:] = (255, 255, 255)

pt1 = (300, 50)
pt2 = (100, 220)
center = (image.shape[1]//2, image.shape[0]//2)
shade = (pt2[0] + 2, pt2[1] + 2)
```

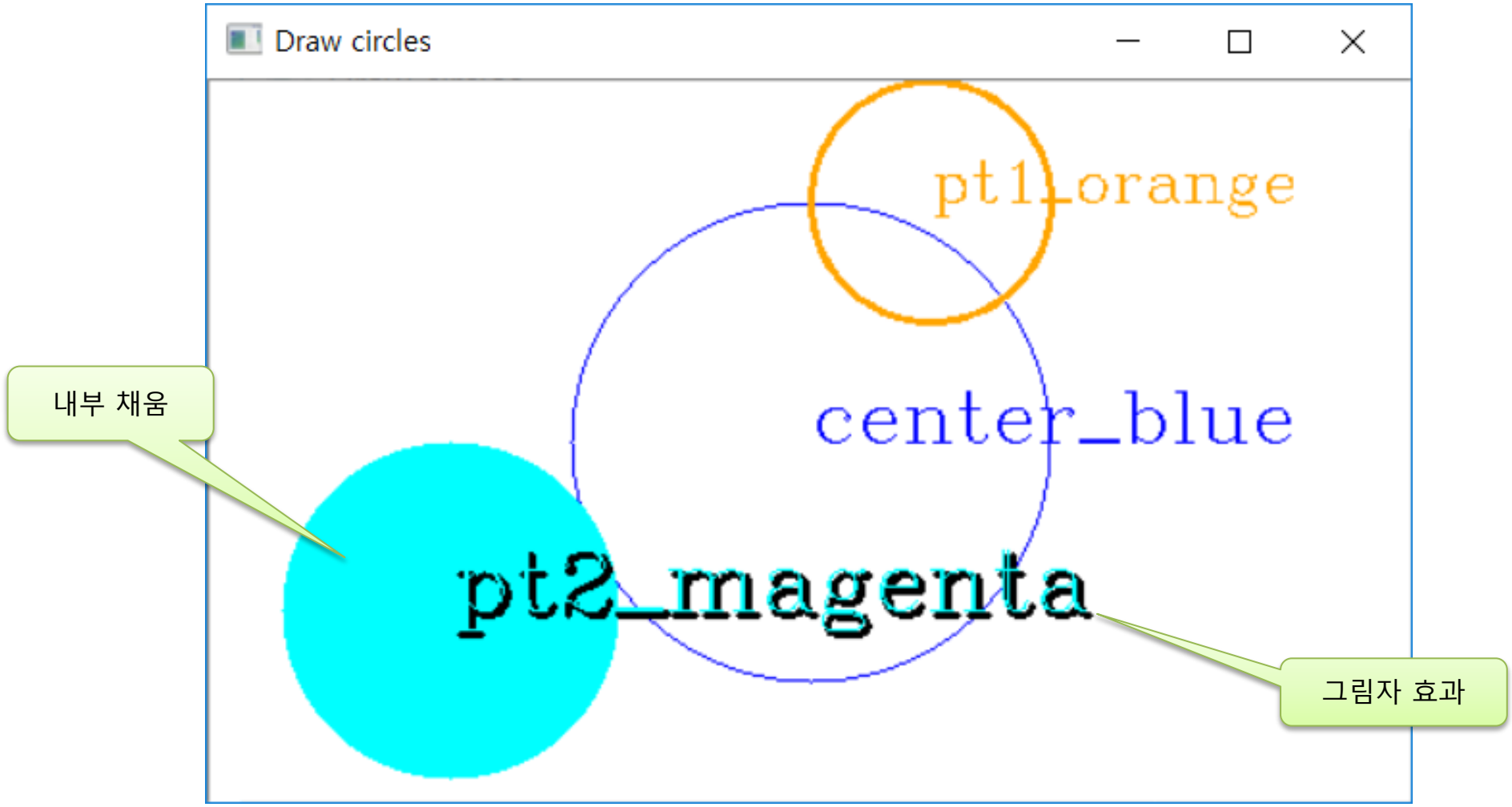
```
cv2.circle(image, center, 100, (255, 0, 0))
cv2.circle(image, pt1, 50, (0, 165, 255))
cv2.circle(image, pt2, 70, (255, 255, 0), -1) # fill

font = cv2.FONT_HERSHEY_COMPLEX
cv2.putText(image, 'center_blue', center, font, 1.0, (255, 0, 0))
cv2.putText(image, 'pt1_orange', pt1, font, 0.8, (0, 165, 255))
cv2.putText(image, 'pt2_cyan', shade, font, 1.2, (0, 0, 0))
cv2.putText(image, 'pt2_cyan', pt2, font, 1.2, (255, 255, 0), 1)

cv2.imshow("Draw circles", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

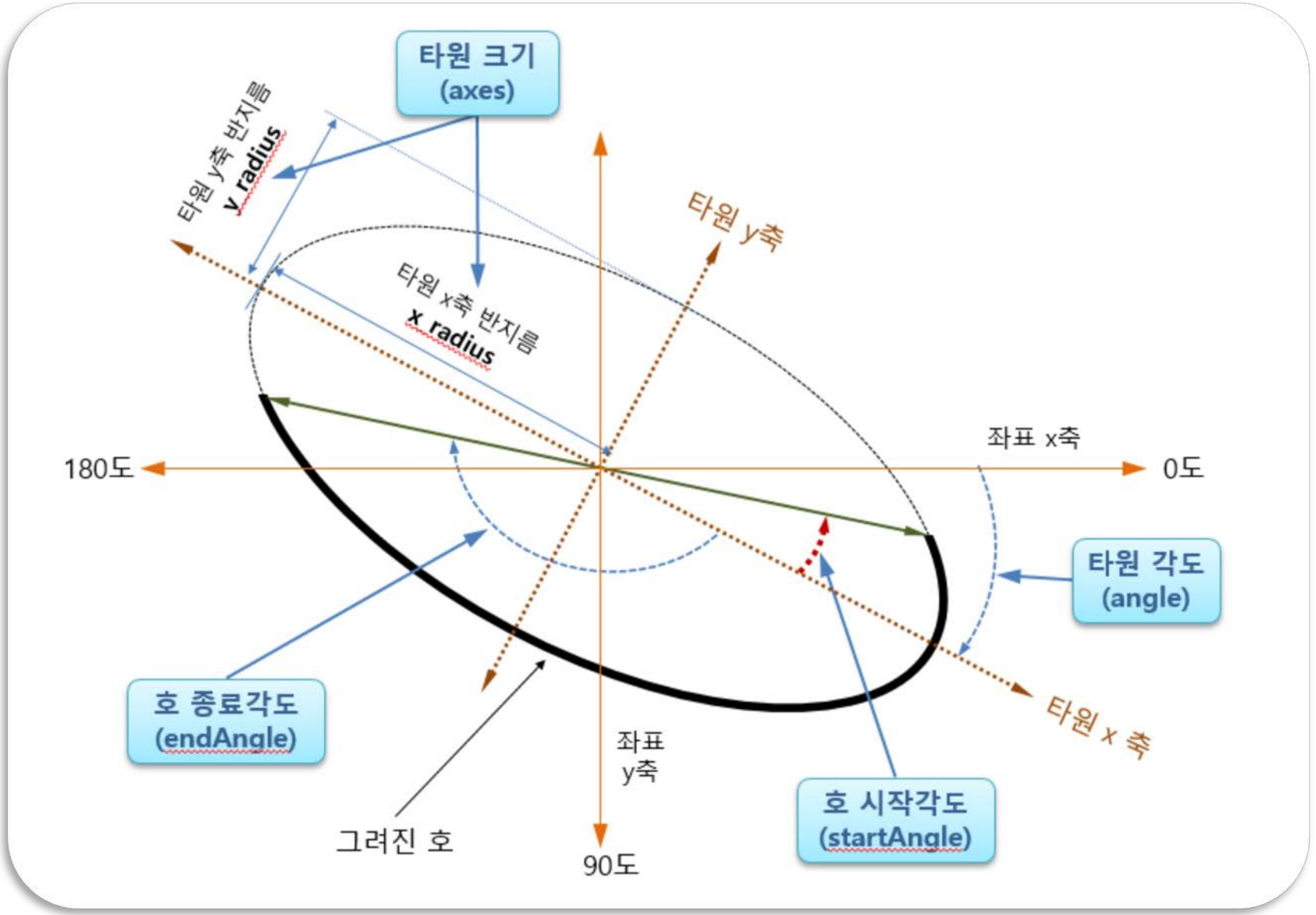
원 그리기

- 실행 결과



타원 그리기

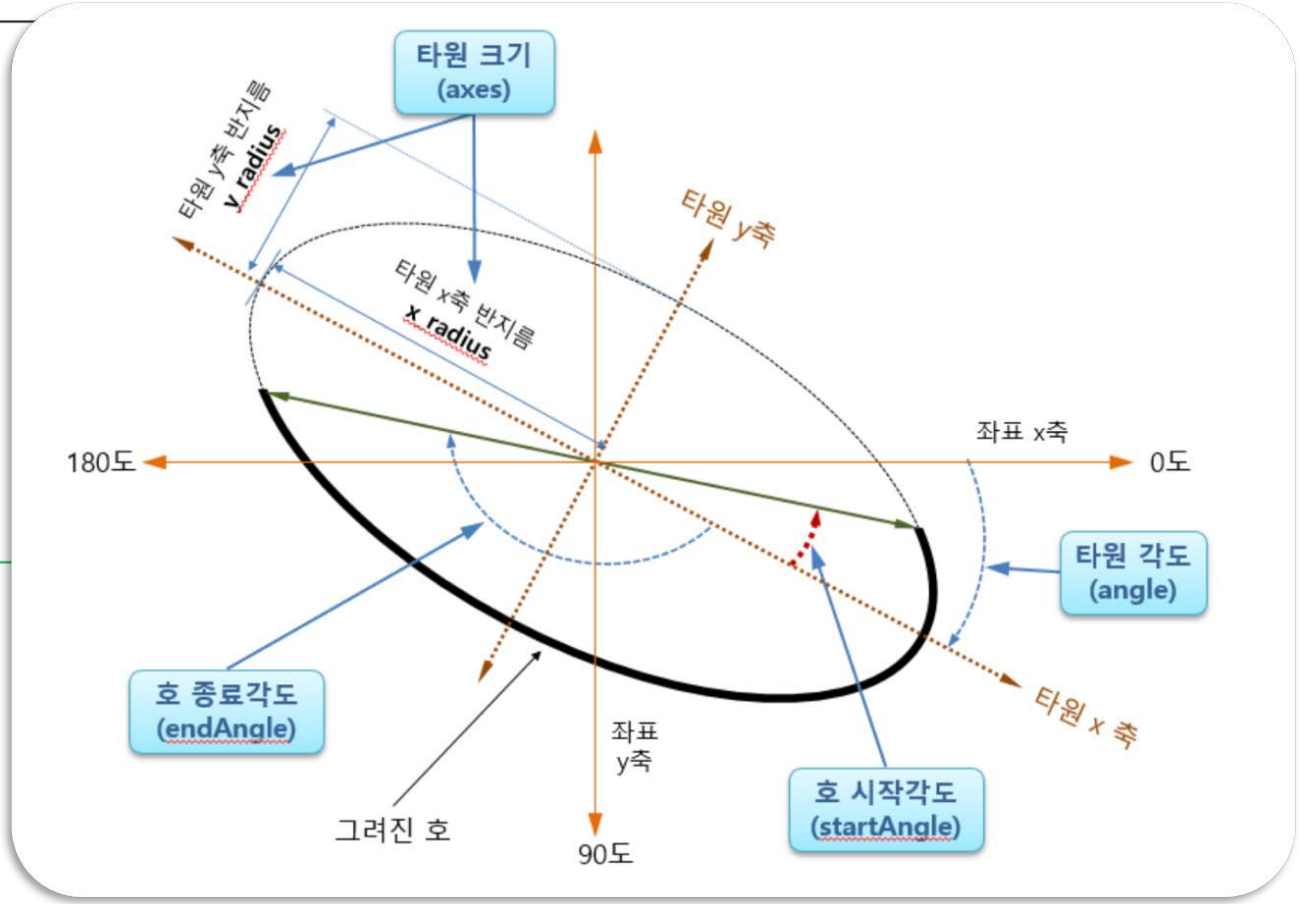
- 타원 그리기 인수 의미



타원 그리기

- **타원 그리기 함수**

함수명과 반환형 및 인수 구조		
cv2.ellipse(img, center, axes, angle, startAngle, endAngle, color[, thickness[, lineType[, shift]]]) →img ■설명: center를 중심으로 axes 크기의 타원을 그린다.		
인수 설명	■img	그릴 대상 행렬(영상)
	■center	원의 중심 좌표
	■axes	타원의 절반 크기(x축 반지름, y축 반지름)
	■angle	타원의 각도 (3시 방향이 0도, 시계방향 회전)
	■startAngle	호의 시작 각도
	■endAngle	호의 종료 각도
	■color	선의 색상
	■thickness	선의 두께
	■lineType	선의 형태
	■shift	좌표에 대한 비트 시프트



타원 그리기

- **타원 그리기 코드**
  - imshow, waitKey destroyAllWindows 제외

```
image = np.zeros((350,700,3[4.3.4 타원 그리기]nt8)
image[:] = (255, 255, 255)

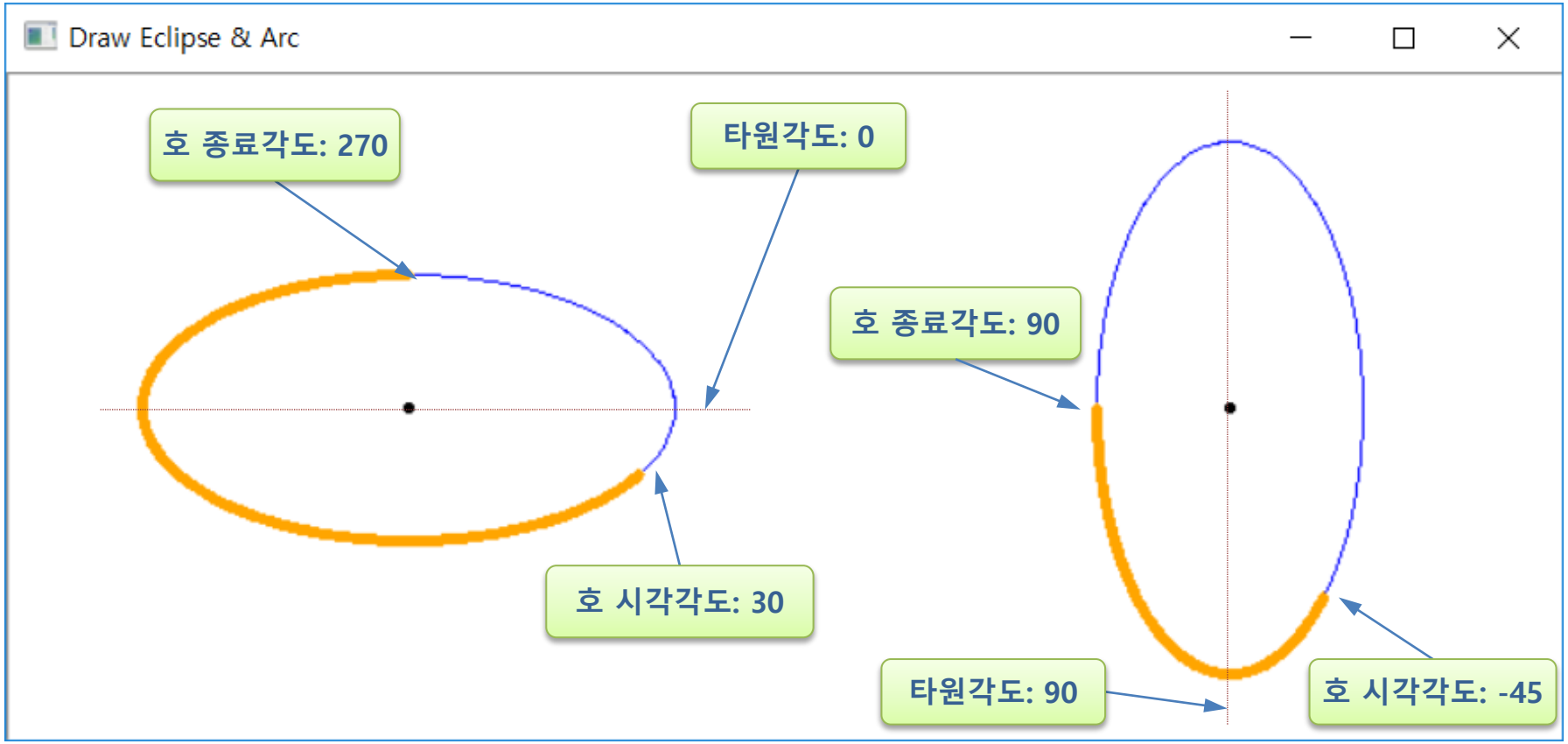
pt1 = (180, 150)
pt2 = (550, 150)
size = (120, 60)

cv2.circle(image, pt1, 1, 0, 2)
cv2.circle(image, pt2, 1, 0, 2)

cv2.ellipse(image, pt1, size, 0, 0, 360, (255, 0, 0), 1)
cv2.ellipse(image, pt1, size, 0, 30, 270, (0, 165, 255), 4)
cv2.ellipse(image, pt2, size, 90, 0, 360, (255, 0, 0), 1)
cv2.ellipse(image, pt2, size, 90, -45, 90, (0, 165, 255), 4)
```

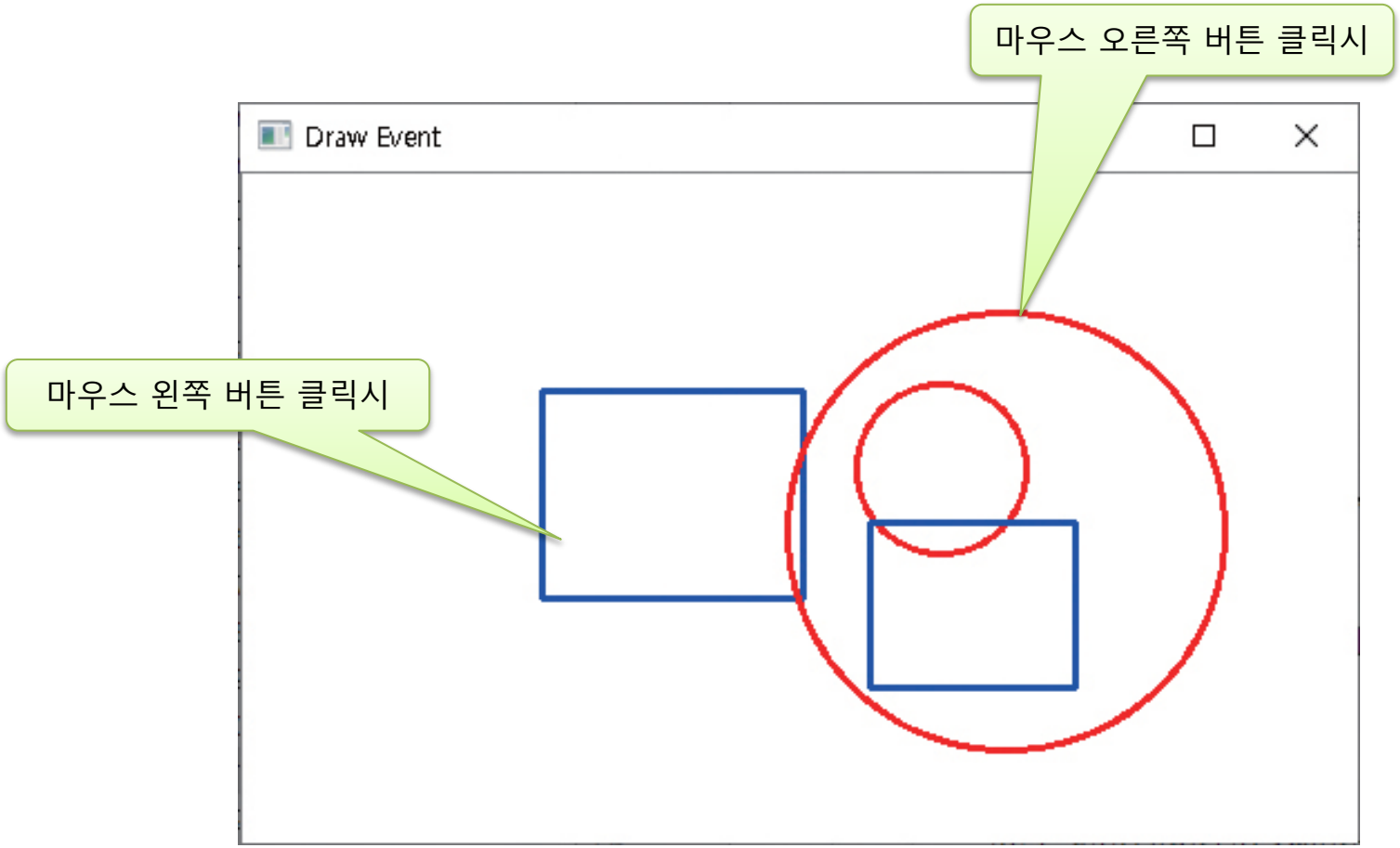
타원 그리기

- 실행 결과


Draw Eclipse & Arc
호 종료각도: 270
타원각도: 0
호 시각각도: 30
호 종료각도: 90
타원각도: 90
호 시각각도: -45

이벤트 & 그리기

- **이벤트 & 그리기 통합 예제**
  - 마우스 첫 클릭 시
    - 왼쪽 : 시작 좌표 저장
    - 오른쪽 : 중심 좌표 저장
  - 마우스 두번 째 클릭 시
    - 왼쪽 : 끝 좌표로 사각형 그리기
    - 오른쪽 : 원 외곽 점으로 원 그리기



이벤트 & 그리기

- **이벤트 & 그리기 통합 코드**
  - 코드 후단 4라인 생략

```
def onMouse(event, x, y, flags, param):
    global title, pt

    if event == cv2.EVENT_LBUTTONDOWN:
        if pt[0] < 0:
            pt = (x, y)
        else:
            cv2.rectangle(image, pt, (x,y), (255, 0, 0), 2)
            cv2.imshow(title, image)
            pt = (-1,-1)
    elif event == cv2.EVENT_RBUTTONDOWN:
        if pt[0] < 0:
            pt = (x, y)
        else:
            dx = pt[0] - x
            dy = pt[1] - y
            radius = int(np.sqrt(dx*dx + dy*dy))
            cv2.[redacted](image, pt, radius, (0, 0, 255), 2)
            cv2.[redacted](title, image)
            pt = (-1,-1)

image = np.zeros((300,500,3), np.uint8)
image[:] = (255, 255, 255)

pt = (-1, -1)
title = "Draw Event"
```

영상 파일 처리

- **영상처리 과정**
  - 영상 파일을 읽어 들여 행렬에 저장
  - 행렬 연산 과정에서 행렬의 원소를 필요할 때마다 눈으로 직접 확인
  - 처리된 결과 행렬을 영상 파일로 저장 →

- **영상파일을 처리해 주는 함수와 활용 방법 필수**

함수 설명		
cv2.imread(filename[, flags]) → retval ■ 설명: 지정한 영상파일로부터 영상을 적재한 후, 행렬로 반환한다.		
인수 설명	■ filename	적재할 영상파일 이름(디렉터리 구조 포함)
	■ flags	적재한 영상을 행렬로 반환될 때 컬러 타입을 결정하는 상수
cv2.imwrite(filename, img[, params]) → retval ■ 설명: img 행렬을 지정한 영상파일로 저장한다.		
인수 설명	■ filename	저장할 영상파일 이름(디렉터리 구조 포함), 확장자명에 따라 영상파일 형식 결정
	■ img	저장하고자 하는 행렬(영상)
	■ params	압축 방식에 사용되는 인수 쌍(paramId, paramValue)

영상 파일 처리

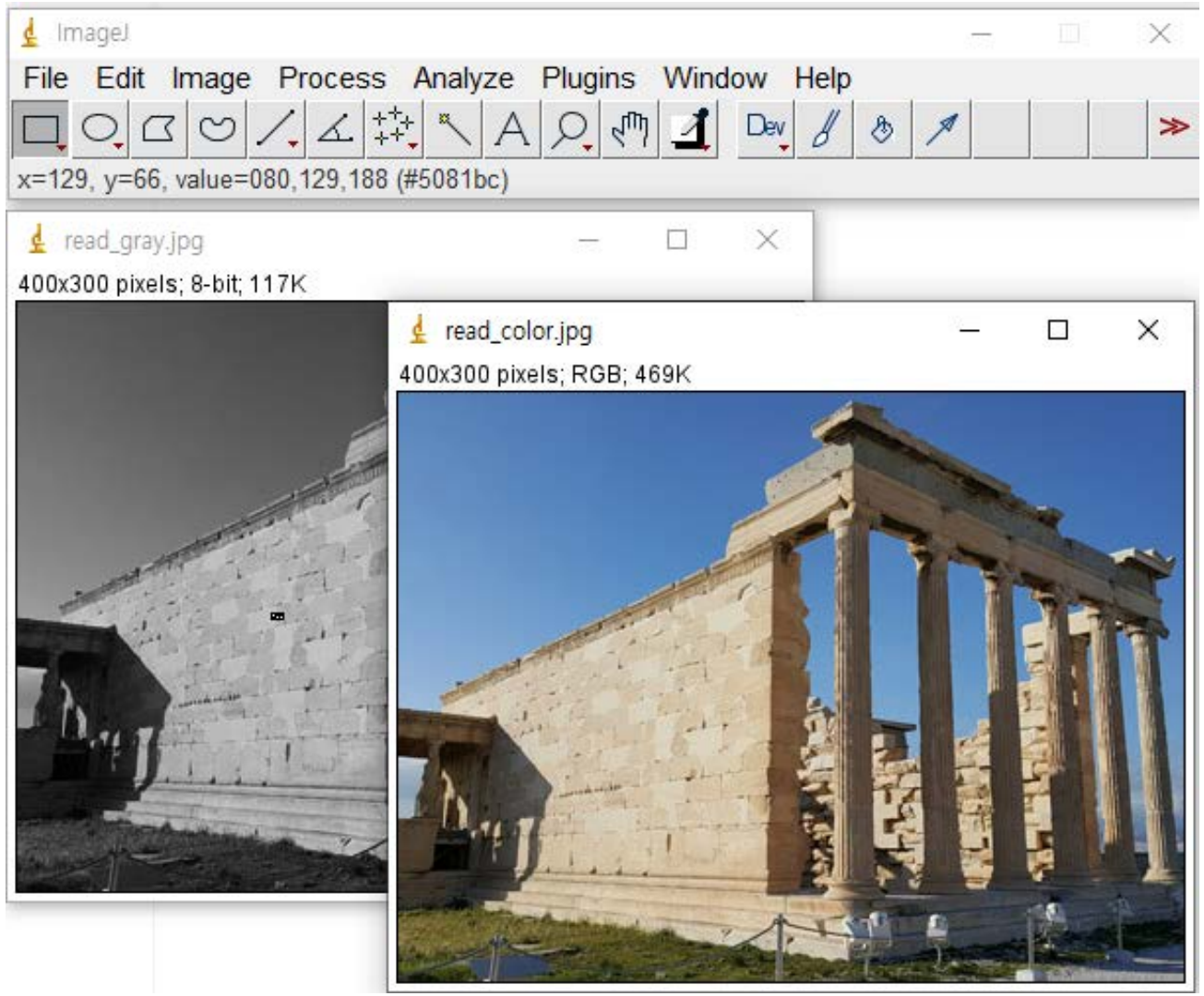
- **색상 유형 결정 상수**

〈표 4.4.1〉 행렬의 컬러 타입 결정 상수

옵션	값	설명
cv2.IMREAD_UNCHANGED	-1	입력 파일에 정의된 타입의 영상을 그대로 반환(알파(alpha) 채널 포함)
cv2.IMREAD_GRAYSCALE	0	명암도(grayscale) 영상으로 변환하여 반환
cv2.IMREAD_COLOR	1	컬러 영상으로 변환하여 반환
cv2.IMREAD_ANYDEPTH	2	입력 파일에 정의된 깊이(depth)에 따라 16비트/32비트 영상으로 변환, 설정되지 않으면 8비트 영상으로 변환
cv2.IMREAD_ANYCOLOR	4	입력 파일에 정의된 타입의 영상을 반환

영상 파일 처리

- **Image 처리 도구**
  - https://imagej.nih.gov/ij/download.html
  - java 기반 이미지 처리/분석 도구
  - 무료 라이센스



영상 파일 처리

- **영상 파일 읽기 코드**
  - read_gray.jpg 파일 필요
  - [학습정보]->[학습자료] 페이지 다운로드
  - .py파일과 동일 위치 저장

```
def print_matInfo(name, image):
    if image.dtype == 'uint8': mat_type = 'CV_8U'
    elif image.dtype == 'int8': mat_type = 'CV_8S'
    elif image.dtype == 'uint16': mat_type = 'CV_16U'
    elif image.dtype == 'int16': mat_type = 'CV_16S'
    elif image.dtype == 'float32': mat_type = 'CV_32F'
    elif image.dtype == 'float64': mat_type = 'CV_64F'
    nchannel = 3 if image.ndim == 3 else 1

    print("%12s: depth(%s), channels(%s) -> mat_type(%sC%d)"
          % (name, image.dtype, nchannel, mat_type, nchannel))

title1 = 'gray2gray'
title2 = 'gray2color'
```

영상 파일 처리

- 영상 파일 읽기 코드

```
gray2gray = cv2.imread("read_gray.jpg", cv2.IMREAD_GRAYSCALE)
gray2color = cv2.imread("read_gray.jpg", cv2.IMREAD_COLOR)

if gray2gray is None or gray2color is None:
    raise Exception("read Error")

print("pixel value : [100, 100]")
print("title1 : ", gray2gray[100,100])
print("title2 : ", gray2color[100,100])

print_matInfo(title1, gray2gray)
print_matInfo(title2, gray2color)

cv2.imshow(title1, gray2gray)
cv2.imshow(title2, gray2color)
cv2.waitKey(0)
```

영상 파일 처리

- 영상 파일 읽기 코드 결과

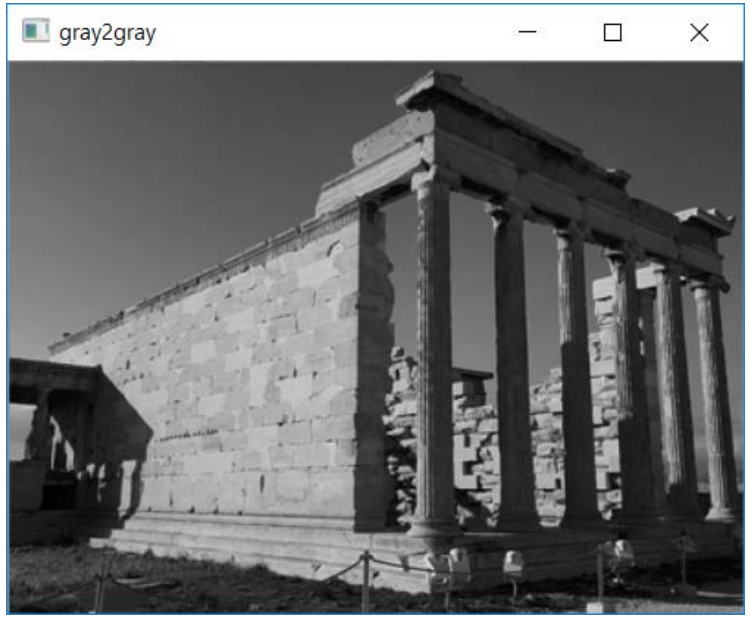
명암도 영상
(1채널 화소)

```
Run:  12.read_image1
C:\Python\python.exe D:/source/chap04/12.read_image1.py
행렬 좌표 (100, 100) 화소값
gray2gray 106
gray2color [106 106 106]

   gray2gray: depth(uint8), channels(1) -> mat_type(CV_8UC1)
  gray2color: depth(uint8), channels(3) -> mat_type(CV_8UC3)
```

3채널 화소: 명암도 영상에서
변환해서 동일한 값임

행렬 자료형





모듈, 패키지

- **파이썬 내장함수**
  - 파이썬에서 미리 만들어 놓은 기본적인 함수들

함수	설명	함수	설명
abs(a)	a의 절댓값 반환	max(a)	객체 a에서 최대 원소값 반환
chr(a)	a 값을 문자 반환	min(a)	객체 a에서 최소 원소값 반환
divmod(a,b)	나눈 몫과 나머지 반환	open()	파일 읽기위한 파일 객체 만듦
enumerate(a)	객체 a의 인덱스와 원소 반환	ord(a)	문자 a의 아스키코드 값 반환
eval(a)	문자열 a를 실행	pow(a, b)	a를 b 제곱한 결과 반환
input()	키보드로 값을 입력받는 함수	range(a,b,c)	c 간격 갖는 범위(a~b)의 객체 반환
int(a)	a를 정수형으로 변환	round(a)	a를 반올림하여 반환
isinstance(a)	객체 a가 클래스의 인스턴스 인지 검사	str(a)	a를 문자열로 반환
len(a)	객체 a의 원소 개수 반환	sum(a)	객체 a 원소값들의 합 반환
list(a)	객체 a를 리스트로 변환	tuple(a)	객체 a를 튜플로 변환
map(int, a)	객체 a의 원소들을 함수 int로 수행한 결과 반환	type(a)	a의 자료형 반환
print()	콘솔창에 결과 출력	zip(a,b ,,,)	여러 객체를 묶어 주는 함수

모듈, 패키지

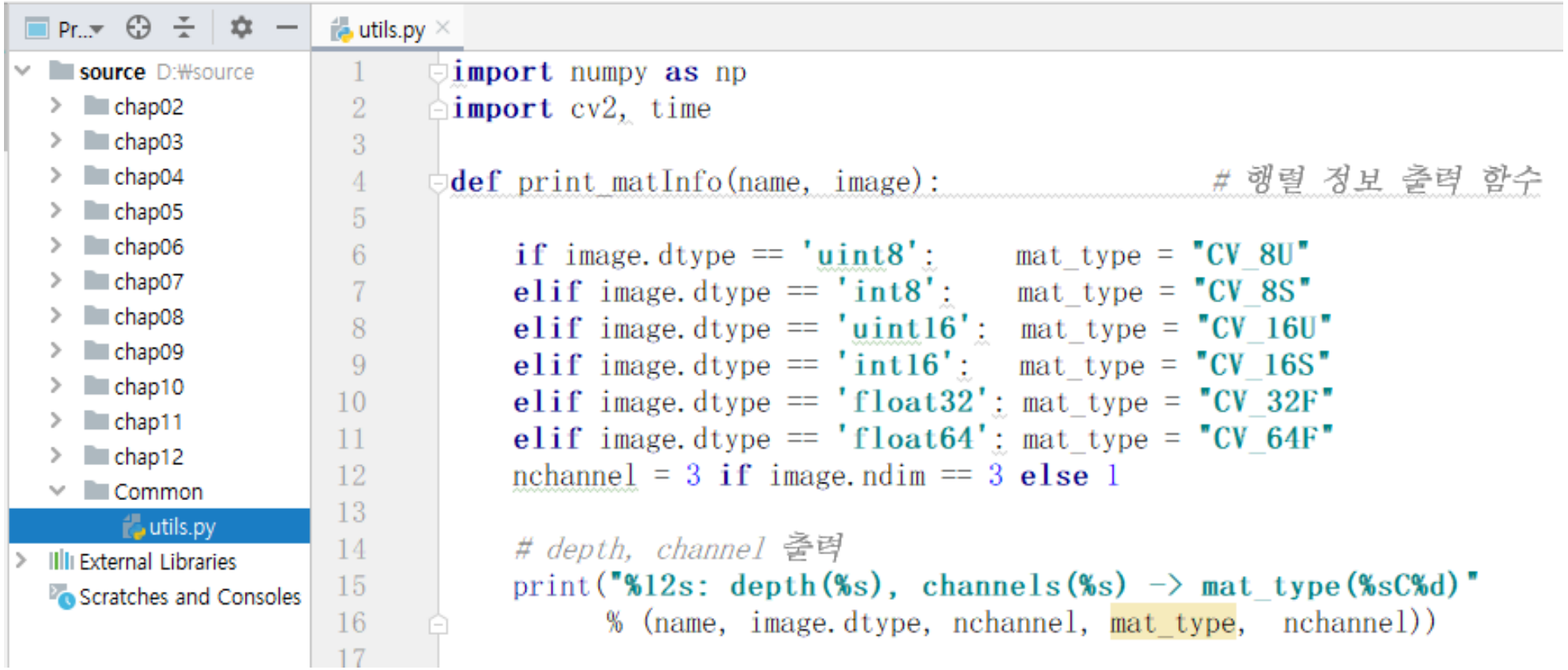
- **컴퓨터 언어 에서 모듈의 의미**
  - 함수들의 모음, 클래스들 의 모음, 라이브러리, 프로그램 등등 각기 정의에 따라 조금씩 다름

- **파이썬에서 모듈**
  - 함수나 변수 또는 클래스를 모아 놓은 파일
  - 다른 파이썬 프로그램에서 불러와 사용할 수 있게 만든 파일

  - 장점
    - 매번 함수를 구현하지 않고 불러와 사용
    - 소스 코드도 간결, 이해 쉬움, 매우 편리

모듈, 패키지

- **모듈 라이브러리 파일 만들기**
  - 프로젝트 루트 폴더(source)에서 마우스 우버튼 눌러 'Common' 이름으로 폴더 생성
    - 루트 폴더에 'Common' 폴더를 두는 것 - 다른 챕터의 소스 파일들에서도 모듈의 쉬운 사용위해
  - 'Common' 폴더에서 파이썬 소스 파일(utils.py) 추가



영상 파일 처리

- **컬러 영상 파일 읽기**
  - read_color.jpg 파일 필요
  - [학습정보]->[학습자료] 페이지 다운로드
  - .py파일과 동일 위치 저장

```python
import cv2
from Common.utils import print_matInfo


title1 = 'color2gray'
title2 = 'color2color'
color2gray = cv2.imread("read_color.jpg", cv2.IMREAD_GRAYSCALE)
color2color = cv2.imread("read_color.jpg", cv2.IMREAD_COLOR)

if color2gray is None or color2color is None:
    raise Exception("read Error")
print("pixel value : [100, 100]")
print("title1 : ", color2gray[100,100])
print("title2 : ", color2color[100,100])


print_matInfo(title1, color2gray)
print_matInfo(title2, color2color)
```

영상 파일 처리

- **실행결과**

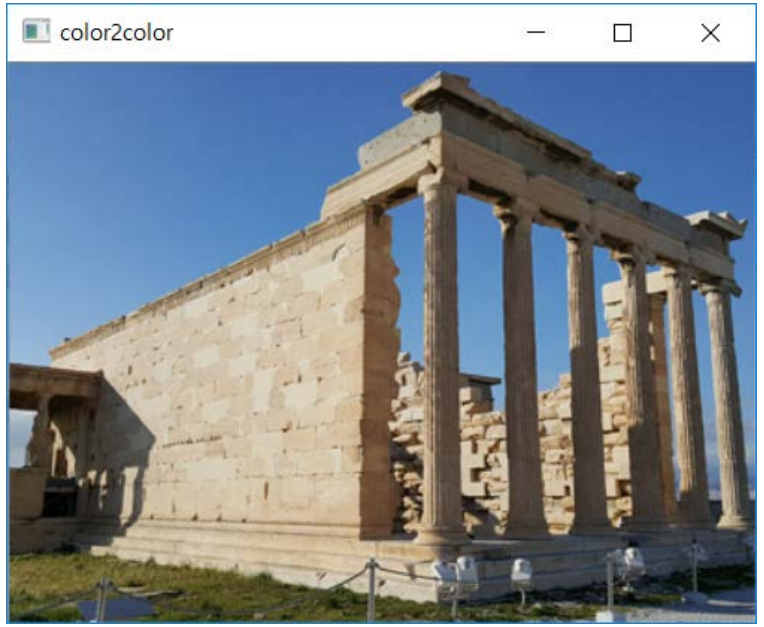
컬러 영상도 명암도 타입으로 읽으면 1채널 영상이 됨

```
Run:  13.read_image2
C:\Python\python.exe D:/source/chap04/13.read_image2.py
행렬 좌표 (100, 100) 화소값
color2gray 137
color2color [197 145  98]

  color2gray: depth(uint8), channels(1) -> mat_type(CV_8UC1)
 color2color: depth(uint8), channels(3) -> mat_type(CV_8UC3)
```

3채널 컬러 영상 화소값





영상 파일 처리

- **16/32 uchar 영상 파일 읽기**
  - read_16.jpg 파일 필요
  - read_32.jpg 파일 필요
  - [학습정보]->[학습자료] 페이지 다운로드
  - .py파일과 동일 위치 저장

```
title1 = '16bit unchanged'
title2 = '32bit unchanged'
color2unchanged1 = cv2.imread("read_16.tif", cv2.IMREAD_UNCHANGED)
color2unchanged2 = cv2.imread("read_32.tif", cv2.IMREAD_UNCHANGED)

if color2unchanged1 is None or color2unchanged2 is None:
    raise Exception("read Error")
print("pixel value : [10, 10]")
print(title1, ":", type(color2unchanged1[10][10][0]), color2unchanged1[10,10])
print(title2, ":", type(color2unchanged2[10][10][0]), color2unchanged2[10,10])

print_matInfo(title1, color2unchanged1)
print_matInfo(title2, color2unchanged2)

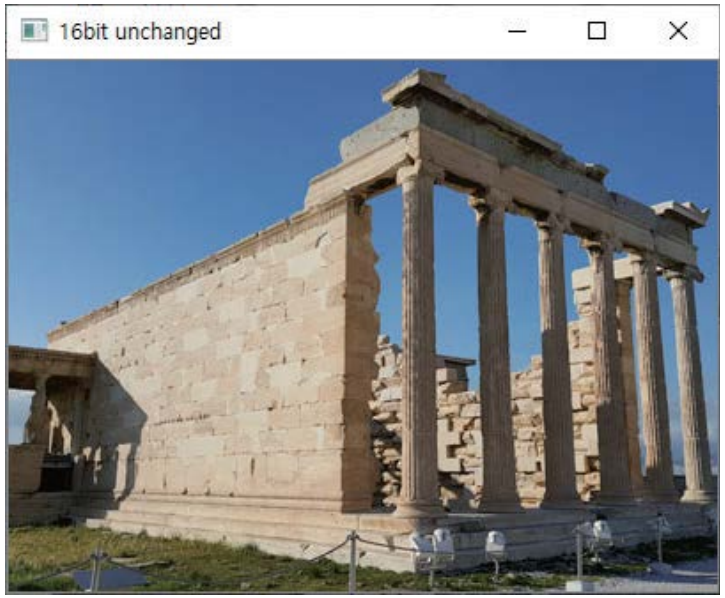
cv2.imshow(title1, color2unchanged1)
cv2.imshow(title2, color2unchanged2)
cv2.waitKey(0)
```

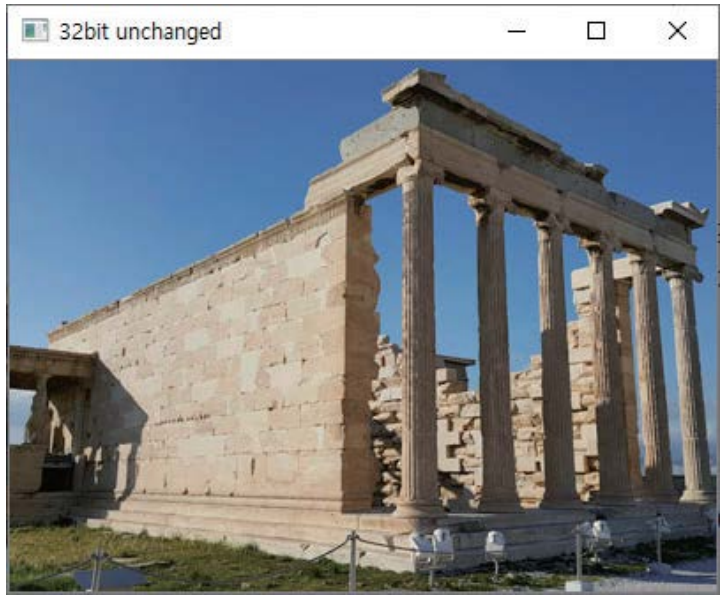
영상 파일 처리

- 실행결과

```
Run:  14.read_image3
C:\Python\python.exe D:/source/chap04/14.read_image3.py
16/32비트 영상 행렬 좌표 (10, 10) 화소값
16bit unchanged 원소 자료형 <class 'numpy.uint16'>
16bit unchanged 화소값(3원소) [48573 34438 23387]
32bit unchanged 원소 자료형 <class 'numpy.float32'>
32bit unchanged 화소값(3원소) [0.7456989  0.52237624 0.36376417]

16bit unchanged: depth(uint16), channels(3) -> mat_type(CV_16UC3)
32bit unchanged: depth(float32), channels(3) -> mat_type(CV_32FC3)
```





영상 파일 저장

- **cv2.imwrite() 함수**
  - 행렬을 영상파일로 저장
  - 확장자에 따라서 JPG, BMP, PNG, TIF, PPM 등의 영상파일 포맷 저장 가능

〈표 4.4.3〉 압축 방식에 사용되는 params 인수 튜플(paramId, paramValue)의 예시

paramId	paramValue (기본값)	설명
cv2.IMWRITE_JPEG_QUALITY	0~100 (95)	JPG 파일 화질, 높은 값일수록 화질 좋음
cv2.IMWRITE_PNG_COMPRESSION	0~9 (3)	PNG 파일 압축 레벨, 높은 값일수록 용량은 적어지고, 압축 시간이 길어짐
cv2.IMWRITE_PXM_BINARY	0 or 1 (1)	PPM, PGM 파일의 이진 포맷 설정

영상 파일 저장

```python
import cv2

image = cv2.imread("read_color.jpg", cv2.IMREAD_COLOR)

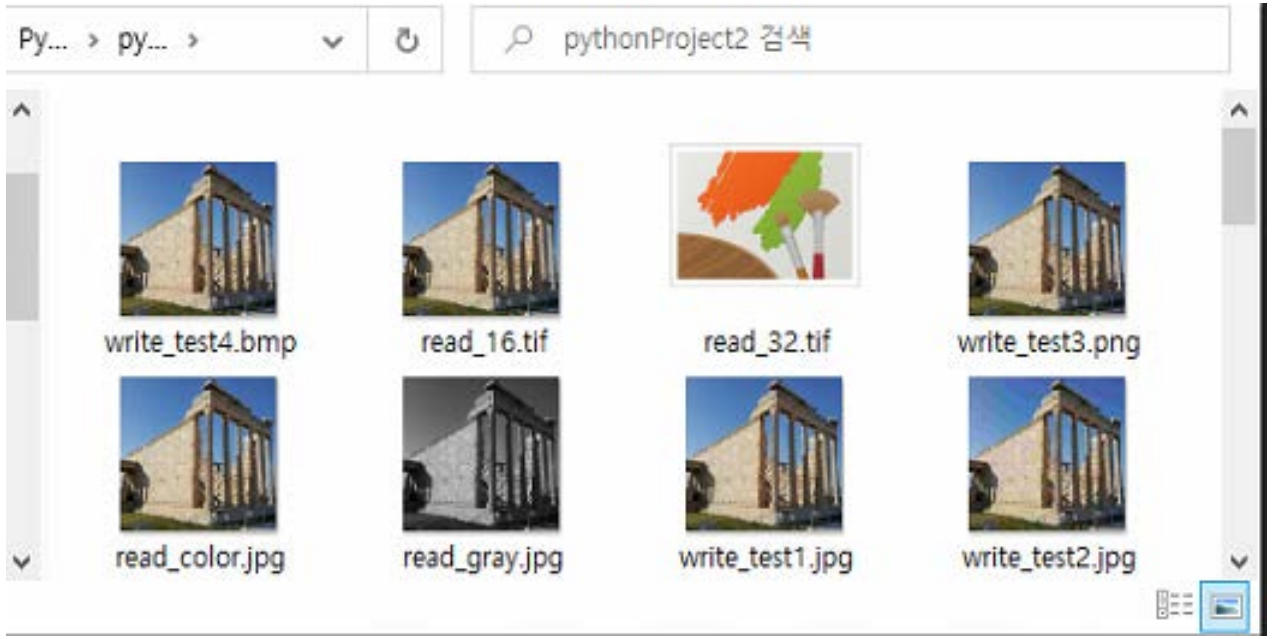
if image is None:
    raise Exception("read Error")

params_jpg = (cv2.IMWRITE_JPEG_QUALITY, 10)
params_png = (cv2.IMWRITE_PNG_COMPRESSION, 9)

cv2.imwrite("write_test1.jpg", image)
cv2.imwrite("write_test2.jpg", image, params_jpg)
cv2.imwrite("write_test3.png", image, params_png)
cv2.imwrite("write_test4.bmp", image)
print('End')
```

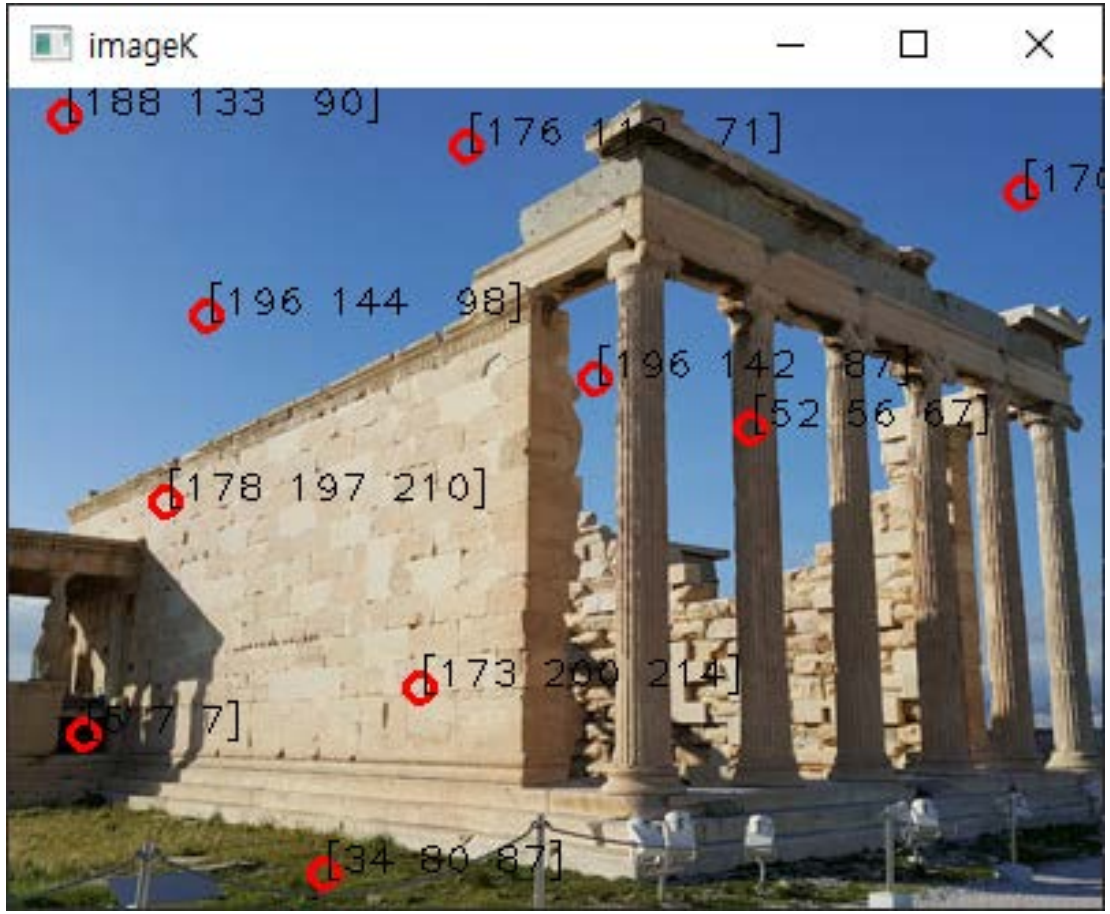
영상 파일 저장

- **실행결과**
  - 크기 및 영상 형식 확인



과제

- **이미지 & 이벤트 처리**
  - color 이미지 읽고 띄우기
  - 마우스 좌측 버튼 클릭시
    - 클릭된 위치를 중심으로 원 그리기
    - 그 점의 화소 값을 글씨로 표시



과제

- **과제 코드 일부**
  - 후단 코드 생략

```
def onMouse(event, x, y, flags, param):
    global title
    if event == cv2.[redacted]:
        cv2.circle(image, [redacted])
        cv2.putText([redacted])
        cv2.imshow(title, image)

title = 'imageK'
image = cv2.imread("read_color.jpg", cv2.IMREAD_COLOR)

if image is None:
    raise Exception("read Error")
```